Better Tooling Interface for ld_audit

Background

- Bad LD_PRELOAD
  - Conflicts
- Rough LD_AUDIT (idea would be a better interface to make it not so rough!)
  - What are the fundamental aspects that would be included in the interface that makes it work better?
  - Isolation
  - All the good stuff! Come on!
- You can use these both for the same problems but just different ways!

Bad LD_PRELOAD

- Env variables where you set it, point it at library.
- Library loaded in front in the same library namespace. All symbols can be mixed and intermixed w the application.
  - Your init() becomes THE init()
- Challenges:
  - Doesnt compose bc if you put multiple things in there that replace the same symbol bad things happen! Tough composability. Symbol pollution. When you link with other applications that uses the same symbol…”KABOOM.”
    -

Rough LD_AUDIT

- Offers a lot on paper
- Set API and get callbacks when interesting events happen.
- (man rtld-audit, link.h)
- When: Libraries loaded, searched (and change these things), symbols are bound, lib namespace separation
    - Symbol resolutions separate!
    - Load 2 copies of libraries with separate namespaces and they dont collide
        - Versions are separate too!
        - Reliably good!
        - Link so many libraries
    - Dont have to make versions of your tool for every version of …“blah blah blah”
- Pthread allocated memory not in the right namespace…
    - Workaround: a little extra code in auditor when you do pthread specific
        - TLS is a challenging thing! Concept not isolated. Thread spec piece of data requires, in your auditor, when you’re binding the thread for pthread set specific from the app or auditor, you need to bind it to the pthread set specific in first auditor to ensure TSL namespace is properly initialized
            - Matt says imagine this next level: imagine this x12…
- Env Variables: DT_AUDIT, How to link w DT_ADUIT
- Keep GOTCHA, interface becomes an implementation of GOTCHA?
- Some customers don’t want to set env variables, would rather have a file
- BUGS are the really big thing:
    - Some things aren’t compatible with AUDIT and crash
    - Capturing workarounds in the new interface

Ben’s Background talk

- Audit bugs fixed in glibc 2.35, RHeL ~9.2+
- Modern OS versions, almost ALL bugs that we know of are fixed!
- Other rough edges
    - Interface uses cookies
        - Intended to be opaque data structures, pointers to linkmaps
        - Want to change it, doubt about change
    - TLS quirks/bugs
        - Recently fixed
        - Reinitialize TLS and all old pointers become invalid (fixed)
        - Pthread set specific and get specific
            - Workaround exists and MUST apply, as theres no good way to fix it via API easily
    - Device Context initialization, teardown
        - Must get the initialization right (ex. CUPT)
        - Always going to be a challenge, must follow techniques
    - Debugging tools that are running auditors
        - Debugging auditors with gdb bugs are fixed only in latest versions of gdb
        - **new fix**
- Things CUPT could do better to work with the ordering?
    - Difficult answer: interface solution, not so much a CUPT problem
- Auditors can’t audit auditors! (soln of composability)
    - Auditor auditing auditor which audits an application, spindling spindling spindle -> incomplete memory map
- Linkmaps are a linked list of linkmaps which references an elf. The elf file could have debug info in it. IF you want debug info,mess with elf!

What We Need:

Few Directions from Matthew:

- New Interface
- Composability
  - Naming Variables
- multiple tools at once
- multiple versions of the same tool, “versioning”
- function wrapping
  - Parameter rewriting
  - Separating things out if they are going to fight, ordering
    - How do things get ordered? Unwrapped?
  - Mangling for c++
  - Trampoline: function wrapping w/o parameters
- Security (some people hide bad stuff)
  - People that care about security disable tools…
- Overheads of PRELOAD and AUDIT
- Replacement solutions
- Introspection and Things like Callbacks and Exits, “CBS”
  - What symbols are present? Versions of symbols?
- Callbacks thread create/destroy, load/unload, at exit, library load/unload
- Detach (restore state, restore and resume, etc.)
- Modifying the Application (muck up symbol bindings to do other things? Inject your stuff into it?)
- Target Users: Tool developers
  - As a cmd line tool!
- Name space for “tools” and name space for “apps”?
  - Does every tool get a namespace? Shared?
  - What happens when we run out?
  - Programmability
- ELF parsing because its pretty easy
- Call APP func
- Unwinding would be out of scope
- Python Layer (doable)
- SDT markers (feeding callbacks!)

Build something new on LD_AUDIT, workarounds and fixes, tool interface that supports composability, function wrapping

Just fix LD_AUDIT, no hiding

What we need: Ben’s list

- Ben wants to bump LD_AUDIT interface for
  - Composibility
    - Init libs. Device
    - Enumerate other tools for ordering
    - Negotiate issues (ex. two things want to collect counters at the same time)
  - bug fixes/workarounds
  - gap filling
    - Ex. (cookies vs. linkmaps)
    - General utility
      - ABI consistency
      - Gotcha should be in there
      - ELF utils, libabigail
      - Finding constructor for the library
      - Mangler / demangler
      - Land on std python markers, named breakpoints
  - Easy interface
    - Quick replacement for LD_PRELOAD
    - For users that like LD_PRELOAD without the “bad”
  - Auditors auditing auditors

Recurring Topics

Many bugs are usually have a workarounds

Jonathan and Chris bring up a lot of problems they had. It seemed like many times there does exist a workaround, which probably belongs in the interface. Furthermore, there were more use cases for Jonathan that would be better suited for GOTCHA.

GOTCHA as an auditor?

JMC

Jonathan: jonathan.madsen@amd.com

Chris: cashton@nvidia.com

Matthew: legendre1@llnl.gov

Ben Woodard: woodard@redhat